浙江大学

本科实验报告

课程名称：　实验 lab2：trace-driven cache simulator

姓　　名：

学　　院：　信息与电子工程学院

系：

专　　业：　信息工程

学　　号：

指导教师：　赵武锋、钟睿

2025 年 12 月 14 日

专业：信息工程
姓名：
学号：
日期：2025.12.14
地点：玉泉校区

浙江大学实验报告

课程名称：计算机设计与组成
指导老师：赵武锋、钟睿
成绩：
实验名称：实验 lab1：trace-driven cache simulator

一、实验目的与任务

1．实现一个踪迹驱动（trace-driven）的 Cache（缓存）模拟器

2．利用 Cache 来评估不同 Cache 架构特性的性能

二、实验过程

（一）实现 trace-driven cache simulator

1．项目说明：

（1）源文件：cache.h：Cache 模拟器定义；cache.c：Cache 模拟器；main.h：模拟驱动程序定义；main.c：模拟驱动程序

（2）测试集：public-block.trace：测试 Cache 块大小；public-assoc.trace：测试 Cache 关联度；public-write.trace：测试 Cache 写策略；public-instr.trace：测试指令 Cache；spice10.trace：spice.trace 的前 10 次访问；；spice100.trace：spice.trace 的前 100 次访问；spice1000.trace：spice.trace 的前 1000 次访问

（3）公开测试的预期输出：public-block1.out；public-assocl.out；public-writel.out；public-instr1.out；spice10.out；spice100.out；spice1000.out

（4）其他文件：Makefile：用于创建模拟器的 makefile；runPublic：用于运行公开测试的 csh 脚本；spice.trace：电路模拟器；cc.trace：C 编译器；tex.trace：Tex 文档处理器

2．Simulator 要求：

（1）必须支持的功能：Total cache size ；Block size ；Unified vs. split I- and D-caches ；Associativity ；Write back vs. write through ；Write allocate vs. write no allocate

（2）模拟器必须跟踪：Number of instruction references；Number of data references ；Number of instruction misses；Number of data misses；Number of words fetched from memory；Number of words copied back to memory

3．主要文件解释：

（1）Makefile 是一个 UNIX make 文件。尝试在复制了文件的本地目录中输入 make 。这将利用提供的程序文件构建模拟器，并生成名为 sim 的可执行文件

（2）main.c 包含模拟器的顶层驱动；cache.c 包含 Cache 模型本身，三个需要编写的模板函数：init_cache()：调用一次，用于构建和初始化 Cache 模型数据结构；perform_access()：在模拟器循环的每次迭代中调用一次，以模拟对 Cache 的单次内存引用；flush()：在模拟结束时调用，用于清除 Cache 内容 。注意，直到所有脏 (dirty) Cache 行（如果有）都被刷出 Cache，并且所有统计数据都因此更新后，模拟才算完成

（3）main.h 是自解释的；cache.h 包含用于初始化和更改 Cache 配置的常量，以及用于实现 Cache 模型的数据结构。

（4）三个用于驱动模拟器的踪迹文件：spice.trace，cc.trace 和 tex.trace 。它们分别代表了大约 100 万次内存引用

4．Cache 构建的具体过程，采用增量式方法

（1）cache 初始化

实现 Cache 的初始化，先考虑最简单的直接映射，并且 i-d 统一的 Unified Cache，即 icache 和 dcache 是同一个；此处需要先编写 init 函数，先初始化所有的统计数据

```
78  /* initialize the cache, and cache statistics data structures */
79  // 初始化统计数据
80  cache_stat_inst.accesses = 0;
81  cache_stat_inst.misses = 0;
82  cache_stat_inst.replacements = 0;
83  cache_stat_inst.demand_fetches = 0;
84  cache_stat_inst.copies_back = 0;
85
86  cache_stat_data.accesses = 0;
87  cache_stat_data.misses = 0;
88  cache_stat_data.replacements = 0;
89  cache_stat_data.demand_fetches = 0;
90  cache_stat_data.copies_back = 0;
```

然后初始化 struct cache 中的相关参数，如 size、n_sets、associativity 等这里特别需要注意的是 offset 和 mask 的计算，由于 cache 拷贝的是一个 block 整体拷贝，因此需要先计算 offset，这一部分包括了 block 中的 word 数量和 word 的字节数，因此这一部分属于块内偏移，然后定义 mask，之后将 addr 和 mask 运算直接就可以提取对应的 index

```
// Unified Cache
icache = dcache = &c1;
cache_usize = cache_usize;
c1.size = cache_usize;
c1.associativity = cache_assoc;
c1.n_sets = cache_usize / (cache_assoc * cache_block_size);
// 计算offset和mask
// offset为块内偏移，由于按块寻找，所以这一部分忽略
c1.index_mask_offset = LOG2(cache_block_size);
// mask为掩码，后续与addr相与即可提取index
c1.index_mask = (c1.n_sets - 1) << c1.index_mask_offset;
```

然后为 LRU 链表分配内存，并且清空内存中的数据

```
  // 分配内存
  c1.LRU_head = (Pcache_line *)malloc(sizeof(Pcache_line) * c1.n_sets);
  c1.LRU_tail = (Pcache_line *)malloc(sizeof(Pcache_line) * c1.n_sets);
  c1.set_contents = (int *)malloc(sizeof(int) * c1.n_sets);
  // 清空
  for (int i = 0; i < c1.n_sets; i++) {
    c1.LRU_head[i] = (Pcache_line)NULL;
    c1.LRU_tail[i] = (Pcache_line)NULL;
    c1.set_contents[i] = 0;
  }
}
```

（2）构建一个统一的、固定块大小、直接映射的 Cache

编写 cache 需要编写 perform_access 函数，首先根据取到的 addr 算出 index 和 tag，addr 会被切分为：tag || index || block offset，根据之前的 mask 得到 index，然后得到 tag

```
  int index, tag;
// 计算index和tag, index = (addr & my_cache.index_mask) >>
  // my_cache.index_mask_offset 地址切分：tag || index || block offset
  // index掩码后移位
  index = (addr & c1.index_mask) >> c1.index_mask_offset;
  // tag 将addr舍弃掉offset和indexs
  tag = addr >> (c1.index_mask_offset + LOG2(c1.n_sets));
```

起始状态整个 cache 是空的，所以需要先新建一个 cache_line，这是 cache 中最小的数据单元，其 struct 的结构在 cache.h 中已经给出，赋值后插入到列表 Cache 中去，然后计算统计数据，由于一开始 Cache 是空的，所以 miss++

```
  // ----------如果set是空的，初始化状态----------
  if (c1.LRU_head[index] == NULL) {
    // 分配内存
    Pcache_line new_cache_line = (Pcache_line)malloc(sizeof(cache_line));
    // 填入数据
    new_cache_line->tag = tag;
    new_line->dirty = FALSE;
    new_cache_line->LRU_next = NULL;
    new_cache_line->LRU_prev = NULL;
    // 将new_cache_line插入到LRU链表
    insert(&c1.LRU_head[index], &c1.LRU_tail[index], new_cache_line);

    // 统计数据
    if (access_type == TRACE_INST_LOAD) {
      cache_stat_inst.accesses++;
      cache_stat_inst.misses++;
      cache_stat_inst.demand_fetches += words_per_block;
    } else {
      cache_stat_data.accesses++;
      cache_stat_data.misses++;
      // 取内存数=取到的words数，每次取一个block
      cache_stat_data.demand_fetches += words_per_block;
    }
  }
```

后续 Cache 不是空的后，需要判断命中和未命中，判断条件是 tag 是否相等，如果相等则命中，需要计算统计数据，如果未命中，则需要计算数据后，准备新的 cache_line，赋值，然后删除掉原来 cache 中的内容，然后 insert 插入新的 cache_line，并且标注 replacement++，进行了替换

```
155     //--------------命中hit--------------------------
156     if (current_line->tag == tag) {
157         if (access_type == TRACE_INST_LOAD)
158           cache_stat_inst.accesses++;
159         else
160           cache_stat_data.accesses++;// hit
161         break;
162       }
163     // -------------未命中miss-------------------------
164     else {
165       // 统计数据
166       if (access_type == TRACE_INST_LOAD) {
167         cache_stat_inst.accesses++;
168         cache_stat_inst.misses++;
169         cache_stat_inst.demand_fetches += words_per_block;
170       } else {
171         cache_stat_data.accesses++;
172         cache_stat_data.misses++;
173         cache_stat_data.demand_fetches += words_per_block;
174       }
175       // 准备新的数据块
176       Pcache_line new_cache_line = (Pcache_line)malloc(sizeof(cache_line));
177       // 填入数据
178       new_cache_line->tag = tag;
179       new_line->dirty = FALSE;
180       new_cache_line->LRU_next = NULL;
181       new_cache_line->LRU_prev = NULL;
182       //替换
183       delete(&c1.LRU_head[index], &c1.LRU_tail[index], c1.LRU_head[index]);
184       insert(&c1.LRU_head[index], &c1.LRU_tail[index], new_cache_line);
185       if (access_type == TRACE_INST_LOAD)
186           cache_stat_inst.replacements++;
187         else
188           cache_stat_data.replacements++;
189       }
190   }
191 }
```

（3）添加可变关联度功能，LRU 替换原则与写回 write_back 功能

基础的 cache 是直接映射的，增加关联性后，即进行分组 set，每个 set 插入多个 cache_line，这些修改仍然在 perform_access 中进行，首先还是要计算 index 和 tag，计算思路和之前一样，然后先处理 Cache 的初始态，即 Cache 中 index 对应的部分是空的，相比基础 cache，由于有 set 的存在，因此需要多一句去判断当前 set 是否已满；由于需要 writeback 功能，所以此时的脏位 dirty 不再是默认的 False，而是要根据 access_type 选择，如果是 store 写入，则 dirty 为需要置 1

```
134     new_line->dirty = (access_type == TRACE_DATA_STORE) ? TRUE : FALSE;
135     new_cache_line->LRU_next = NULL;
136     new_cache_line->LRU_prev = NULL;
```

```
128   // -----------如果set是空的，初始化状态------------
129   if (c1.LRU_head[index] == NULL) {
130     // 分配内存
131     Pcache_line new_cache_line = (Pcache_line)malloc(sizeof(cache_line));
132     // 填入数据
133     new_cache_line->tag = tag;
134     new_line->dirty = (access_type == TRACE_DATA_STORE) ? TRUE : FALSE;
135     new_cache_line->LRU_next = NULL;
136     new_cache_line->LRU_prev = NULL;
137     // 将new_cache_line插入到LRU链表
138     insert(&c1.LRU_head[index], &c1.LRU_tail[index], new_cache_line);
139     // set内容数+1，总的内容数量+1，以判断该set是否已满员
140     c1.set_contents[index]++;
141     c1.contents++;
```

之后处理 Cache 非空状态，由于 set 中有多个 cache_line，相比之前直接比较 tag，此时需要遍历整个 index 对应的 set 部分的所有 cache_line，如果 tag 相等则赋值给 target 表明 hit 命中了，然后分为 hit 和 miss 分别处理

```
157   else {
158     Pcache_line current_line = c1.LRU_head[index];
159     Pcache_line target_line = (Pcache_line)NULL;
160
161     // 遍历链表查找
162     while (current_line != (Pcache_line)NULL) {
163       if (current_line->tag == tag) {
164         target_line = current_line; // hit
165         break;
166       }
167       current_line = current_line->LRU_next; // 继续遍历
168     }
```

Hit 统计数据，由于需要写回功能，需要修改此时的 dirty 值，如果 access_type 属于 store 写入指令，则 dirty 置 1，由于 replacement 使用了 LRU，此时需要将最新的 block 放在链表的 head 位，表示最新最近访问，然后删除掉链表的 tail 位，这个位置是最远最久没访问的位置

```
169     // ------------命中hit,此时target_line不为NULL-----
170     if (target_line != (Pcache_line)NULL) {
171       // 更新统计
172       if (access_type == TRACE_INST_LOAD)
173         cache_stat_inst.accesses++;
174       else
175         cache_stat_data.accesses++;
176
177       // 处理写操作 (Dirty Bit)
178       if (access_type == TRACE_DATA_STORE)
179         target_line->dirty = TRUE;
180
181       // LRU 更新：将命中行移至头部
182       delete (&c1.LRU_head[index], &c1.LRU_tail[index], target_line);
183       insert(&c1.LRU_head[index], &c1.LRU_tail[index], target_line);
184     }
```

Miss 时，先统计 access、miss 等数据，然后比 base 不同之处为，此时需要判断 set 是否满员，如果没满员直接插入 set 的下一个位置即可，如果满员，则需要根据 LRU 原则去替换，由于此时采用的是 write_back 方式，只有当需要替换时，修改的 cache_line 需要放回，此时才需要去判断 dirty 位，如果为 1，则 copies_back += words_per_block

```
185     // --------------未命中miss--------------------------
186     else {
187       // 统计数据
188       if (access_type == TRACE_INST_LOAD) {
189         cache_stat_inst.accesses++;
190         cache_stat_inst.misses++;
191         cache_stat_inst.demand_fetches += words_per_block;
192       } else {
193         cache_stat_data.accesses++;
194         cache_stat_data.misses++;
195         cache_stat_data.demand_fetches += words_per_block;
196       }
197       // 准备新的数据块
198       Pcache_line new_cache_line = (Pcache_line)malloc(sizeof(cache_line));
199       // 填入数据
200       new_cache_line->tag = tag;
201       new_line->dirty = (access_type == TRACE_DATA_STORE) ? TRUE : FALSE;
202       new_cache_line->LRU_next = NULL;
203       new_cache_line->LRU_prev = NULL;
```

```
204       // set已满员时,删除掉原来的数据,进行替换
205       if (c1.set_contents[index] >= c1.associativity) {
206         // 统计replacement
207         if (access_type == TRACE_INST_LOAD)
208           cache_stat_inst.replacements++;
209         else
210           cache_stat_data.replacements++;
211         // 定义链表尾部
212         Pcache_line victim = c1.LRU_tail[index];
213         // 检查写回Write Back逻辑，Cache和Mem交互
214         if (victim->dirty) {
215           if (access_type == TRACE_INST_LOAD)
216             cache_stat_inst.copies_back += words_per_block;
217           else
218             cache_stat_data.copies_back += words_per_block;
219         }
220         // 删除
221         delete (&c1.LRU_head[index], &c1.LRU_tail[index], victim);
222         // 释放内存，防止泄漏
223         free(victim);
224         insert(&c1.LRU_head[index], &c1.LRU_tail[index], new_cache_line);
225       }
226       // set未满员时，直接插入到头部
227       else {
228         // 将new_cache_line插入到LRU链表
229         insert(&c1.LRU_head[index], &c1.LRU_tail[index], new_cache_line);
230         c1.set_contents[index]++;
231         c1.contents++;
232       }
```

（4）添加 flush 功能

增加写入功能后，需要补充 flush，在 cache 模拟结束后，需要将 dirty 为 1 的 Cache_line 写会到内存中，并且更新统计信息，释放内存

```
239  void flush() {
240    int i;
241    Pcache_line current;
242    Pcache_line next;
243
244    // 遍历所有的 Set
245    for (i = 0; i < c1.n_sets; i++) {
246      current = c1.LRU_head[i];
247
248      // 遍历该 Set 的链表
249      while (current != (Pcache_line)NULL) {
250        // 检查脏位
251        if (current->dirty == TRUE) {
252          // 如果是脏的，强制写回，增加统计计数
253          cache_stat_data.copies_back += words_per_block;
254        }
255
256        // 释放内存
257        next = current->LRU_next;
258        free(current);
259        current = next;
260      }
261
262      c1.LRU_head[i] = NULL;
263      c1.LRU_tail[i] = NULL;
264      c1.set_contents[i] = 0;
265    }
266  }
```

（5）添加分离式 (Split) 组织结构功能

之前一直采用的是 Unified Cache，即不区分 instruction 和 data 所使用的 Cache，此处通过修改之前设计的 cache 实现根据 cache_split 决定是否区分，只需要多写一个 split 的 cache 即可，然后根据 ifelse 判断语句决定使用的 cache 类型

由于用到了多个 cache，所以先写一个辅助函数，先初始化一个 cache 的函数

```
76  void init_one_cache(Pcache c, int size) {
77    c->size = size;
78    c->associativity = cache_assoc;
79    c->n_sets = size / (cache_assoc * cache_block_size);
80
81    c->index_mask_offset = LOG2(cache_block_size);
82    c->index_mask = (c->n_sets - 1) << c->index_mask_offset;
83
84    c->LRU_head = (Pcache_line *)malloc(sizeof(Pcache_line) * c->n_sets);
85    c->LRU_tail = (Pcache_line *)malloc(sizeof(Pcache_line) * c->n_sets);
86    c->set_contents = (int *)malloc(sizeof(int) * c->n_sets);
87    c->contents = 0;
88
89    for (int i = 0; i < c->n_sets; i++) {
90      c->LRU_head[i] = NULL;
91      c->LRU_tail[i] = NULL;
92      c->set_contents[i] = 0;
93    }
94  }
```

然后根据 cache_split，如果为 1，则需要分别初始化 c1 和 c2 两个 Cache，如果为 0 则保持原来逻辑，icache 和 dcache 同时指向 c1

```
  // 分离式 Cache (Split)
  if (cache_split) {
    icache = &c1;
    dcache = &c2;
    init_one_cache(icache, cache_isize);
    init_one_cache(dcache, cache_dsize);
  }
  // 统一式 Cache (Unified)
  else {
    icache = &c1;
    dcache = &c1; // 指向同一个结构体
    init_one_cache(icache, cache_usize);
  }
}
```

然后编写 perform_access 时，只需要保持原来逻辑不变即可，最多需要修改统计数据的逻辑，如果是原来的 Unified，则指令和数据的统计数据没必要区分，但是此时 split 还是需要区分的

改写 split 后，同时需要改写的还有 flush 函数，和初始化一样，先设计一个辅助函数处理一个 cache 的 flush，然后根据 split 决定 flush 指令还是数据

```
/************************************************************/
/* 辅助函数：清理单个 Cache */
void flush_one_cache(Pcache c, Pcache_stat stat) {
  int i;
  Pcache_line current;
  Pcache_line next;

  for (i = 0; i < c->n_sets; i++) {
    current = c->LRU_head[i];
    while (current != NULL) {
      if (current->dirty == TRUE) {
        stat->copies_back += words_per_block;
      }
      next = current->LRU_next;
      free(current);
      current = next;
    }
    c->LRU_head[i] = NULL;
    c->LRU_tail[i] = NULL;
    c->set_contents[i] = 0;
  }
}

void flush() {
  /* flush the cache */
  if (cache_split) {
    flush_one_cache(icache, &cache_stat_inst);
    flush_one_cache(dcache, &cache_stat_data);
  } else {
    // Unified Cache: 所有的 Dirty 都是 Data store 造成的
    flush_one_cache(icache, &cache_stat_data);
  }
}
/************************************************************/
```

（6）添加写直达 (Write through) 写策略功能

相比 writeback，写直达策略在写操作时，直接将数据写回到内存中，而写回策略在写操作时只是将数据写回到 cache 中，只有在 cache 块被替换时，才会将数据写回到内存中。在代码上体现在，处理 dirty 位的逻辑，此时需要判断如果是 write_through，则不需要标记脏位，copies_back += 1 即可，如果是 write_back 则保持之前的逻辑

```
136   // ----------------命中 HIT------------------
137   if (target_line != (Pcache_line)NULL) {
138     // 更新统计
139     if (access_type == TRACE_INST_LOAD)
140       cache_stat_inst.accesses++;
141     else
142       cache_stat_data.accesses++;
143
144     // 处理写操作
145     if (access_type == TRACE_DATA_STORE) {
146       if (cache_writeback) {
147         // [Write-Back]: 只标记脏，不写内存
148         target_line->dirty = TRUE;
149       } else {
150         // [Write-Through]: 立即写内存
151         cache_stat_data.copies_back += 1;
152         target_line->dirty = FALSE;
153       }
154     }
155
156     // LRU 更新：将命中行移至头部
157     delete(&c1.LRU_head[index], &c1.LRU_tail[index], target_line);
158     insert(&c1.LRU_head[index], &c1.LRU_tail[index], target_line);
159   }
```

```
194   // set已满员时,删除掉原来的数据,进行替换
195   if (c1.set_contents[index] >= c1.associativity) {
196     // 统计replacement
197     if (access_type == TRACE_INST_LOAD)
198       cache_stat_inst.replacements++;
199     else
200       cache_stat_data.replacements++;
201
202     // 定义链表尾部
203     Pcache_line victim = c1.LRU_tail[index];
204
205     // 检查写回 (Eviction),Write-Through 模式下 dirty,永远为FALSE，所以这里不会触发
206     if (victim->dirty) {
207       if (access_type == TRACE_INST_LOAD)
208         cache_stat_inst.copies_back += words_per_block;
209       else
210         cache_stat_data.copies_back += words_per_block;
211     }
212
213     // 删除并释放
214     delete(&c1.LRU_head[index], &c1.LRU_tail[index], victim);
215     free(victim);
216   }
```

还有一个小小的修改是，注意到其实没必要单独处理 cache 是空的，因为起始的 cache 空的状态和之后 miss 的状态是等价的，因此删除了第一部分的冗余代码

（7）添加写不分配 (Write no-allocate) 分配策略功能

写不分配策略和写分配策略的主要区别在于，写不分配策略在写操作时，如果 cache 未命中，则直接将数据写回到内存中，而不分配 cache 块。而写分配策略在写操作时，如果 cache 未命中，则分配一个 cache 块，并将数据写回到 cache 块中。

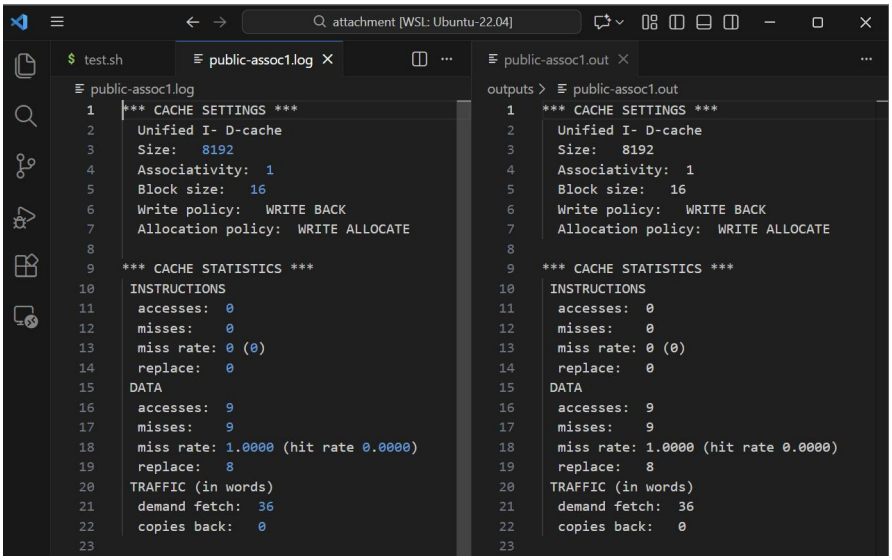
主要体现在 miss 状态的处理，如果是写缺失且配置为“写不分配”，则直接写内存，不进入 Cache，此时直接 return 跳出函数即可，剩下的对 cache 的操作不会进行，如果是 Load Miss 或者 Store Miss + Write Allocate 才会继续

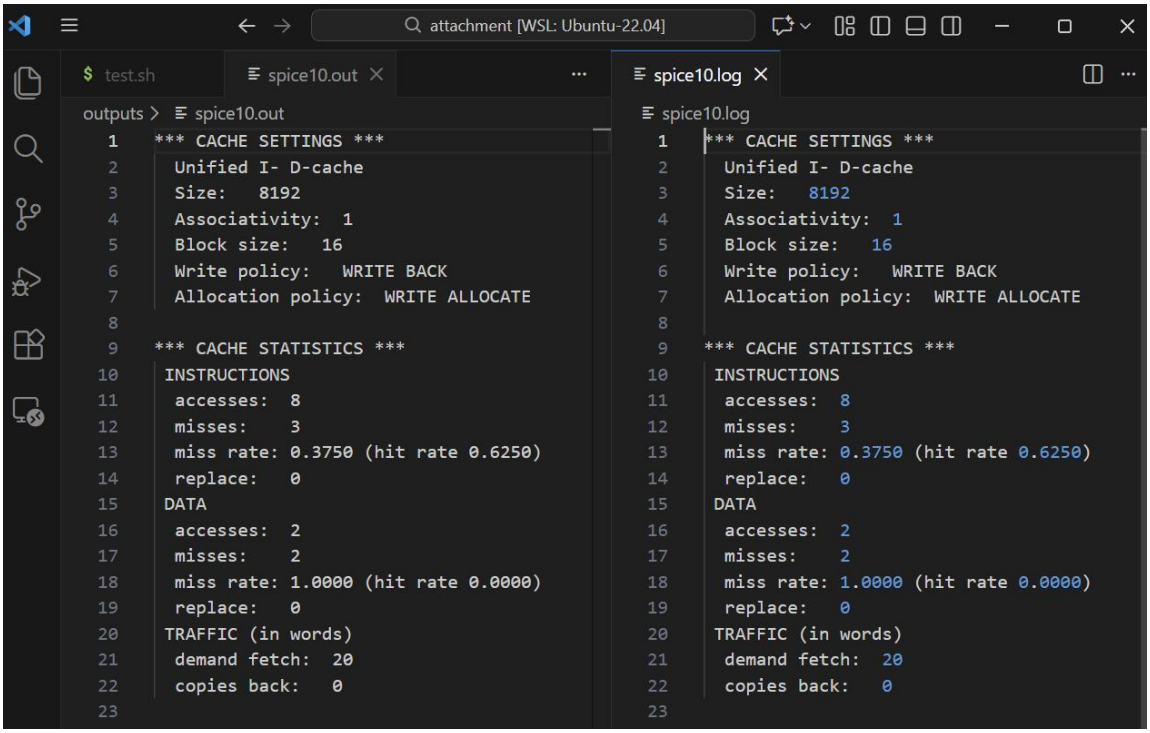
```
176    // ------------------------缺失 (MISS) ----------------------
177    else {
178      current_stat->accesses++;
179      current_stat->misses++;
180
181      // [Write No-Allocate 逻辑]
182      if (access_type == TRACE_DATA_STORE && !cache_writealloc) {
183        current_stat->copies_back += 1;
184        return; // 直接返回
185      }
186
187      // Fetch, 只有 Load Miss 或者 Store Miss + Write Allocate 才继续
188      current_stat->demand_fetches += words_per_block;
```

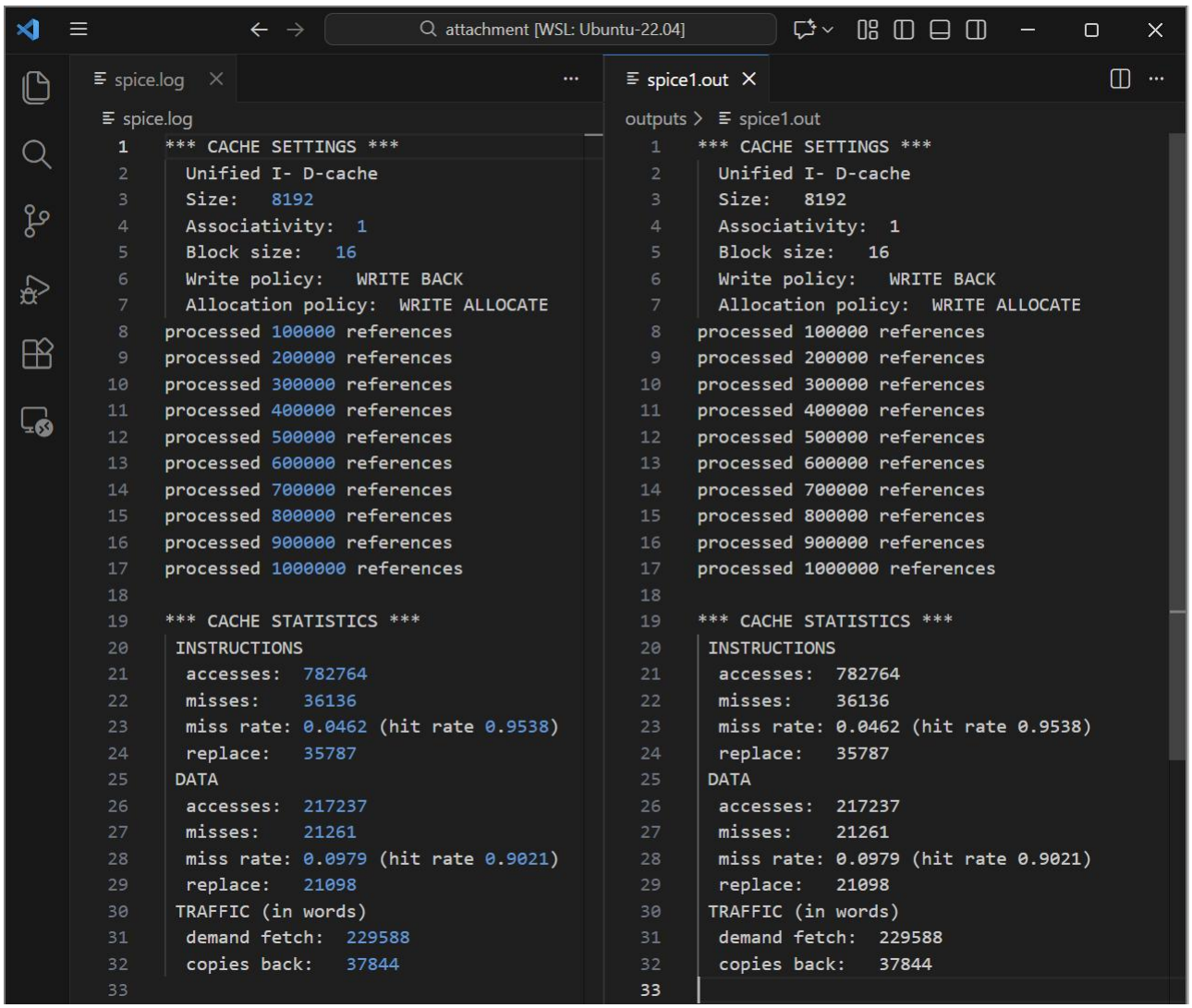
5．构建 Cache 的测试

（1）进入 wsl 后，在 code 目录下运行 make，生成 sim 文件

（2）然后执行 runPublic 下的命令，由于系统的原因我选择了依次执行，得到的效果是一致的，分析 log 和 out 的日志记录，可以看出，是符合预期输出结果的




attachment [WSL: Ubuntu-22.04]
test.sh
spice10.out
outputs > spice10.out
*** CACHE SETTINGS ***
Unified I- D-cache
Size: 8192
Associativity: 1
Block size: 16
Write policy: WRITE BACK
Allocation policy: WRITE ALLOCATE
*** CACHE STATISTICS ***
INSTRUCTIONS
accesses: 8
misses: 3
miss rate: 0.3750 (hit rate 0.6250)
replace: 0
DATA
accesses: 2
misses: 2
miss rate: 1.0000 (hit rate 0.0000)
replace: 0
TRAFFIC (in words)
demand fetch: 20
copies back: 0
spice10.log
spice10.log
*** CACHE SETTINGS ***
Unified I- D-cache
Size: 8192
Associativity: 1
Block size: 16
Write policy: WRITE BACK
Allocation policy: WRITE ALLOCATE
*** CACHE STATISTICS ***
INSTRUCTIONS
accesses: 8
misses: 3
miss rate: 0.3750 (hit rate 0.6250)
replace: 0
DATA
accesses: 2
misses: 2
miss rate: 1.0000 (hit rate 0.0000)
replace: 0
TRAFFIC (in words)
demand fetch: 20
copies back: 0


attachment [WSL: Ubuntu-22.04]
spice.log
spice.log
*** CACHE SETTINGS ***
Unified I- D-cache
Size: 8192
Associativity: 1
Block size: 16
Write policy: WRITE BACK
Allocation policy: WRITE ALLOCATE
processed 100000 references
processed 200000 references
processed 300000 references
processed 400000 references
processed 500000 references
processed 600000 references
processed 700000 references
processed 800000 references
processed 900000 references
processed 1000000 references
*** CACHE STATISTICS ***
INSTRUCTIONS
accesses: 782764
misses: 36136
miss rate: 0.0462 (hit rate 0.9538)
replace: 35787
DATA
accesses: 217237
misses: 21261
miss rate: 0.0979 (hit rate 0.9021)
replace: 21098
TRAFFIC (in words)
demand fetch: 229588
copies back: 37844
spice1.out
outputs > spice1.out
*** CACHE SETTINGS ***
Unified I- D-cache
Size: 8192
Associativity: 1
Block size: 16
Write policy: WRITE BACK
Allocation policy: WRITE ALLOCATE
processed 100000 references
processed 200000 references
processed 300000 references
processed 400000 references
processed 500000 references
processed 600000 references
processed 700000 references
processed 800000 references
processed 900000 references
processed 1000000 references
*** CACHE STATISTICS ***
INSTRUCTIONS
accesses: 782764
misses: 36136
miss rate: 0.0462 (hit rate 0.9538)
replace: 35787
DATA
accesses: 217237
misses: 21261
miss rate: 0.0979 (hit rate 0.9021)
replace: 21098
TRAFFIC (in words)
demand fetch: 229588
copies back: 37844

进一步运行 diff 指令，确定了所有的 log 文件和预期输出 out 是完全一致的



（3）根据 sim.pdf 的指导下，我还编写了 test 文件去对比进一步确定 cache 的正确性

Sim 指导书中的表格也是一种测试文件

						Instructions		Data		Total	
CS	I- vs D-	BS	Ass	Write	Alloc	Misses	Repl	Misses	Repl	DF	CB
8 K	Split	16	1	WB	WA	24681	24173	8283	7818	131856	12024
16 K	Split	16	1	WB	WA	11514	10560	5839	5051	69412	8008
32 K	Split	16	1	WB	WA	5922	4321	1567	520	29956	3628
64 K	Split	16	1	WB	WA	2619	484	1290	103	15636	3324
8 K	Unified	16	1	WB	WA	36136	35787	21261	21098	229588	37844
8 K	Unified	32	1	WB	WA	26673	26502	19561	19476	369872	69104
8 K	Unified	64	1	WB	WA	23104	23029	20377	20324	695696	136112
32 K	Split	128	1	WB	WA	1964	1726	459	280	77536	7296
8 K	Split	64	2	WB	WA	6590	6462	3160	3032	156000	18880
8 K	Split	64	4	WB	WA	6025	5897	875	747	110400	7296
8 K	Split	64	8	WB	WA	6435	6307	803	675	115808	6656
8 K	Split	64	16	WB	WA	6536	6408	799	671	117360	6624
8 K	Split	64	128	WB	WA	6523	6395	790	662	117008	6576
1 K	Split	64	2	WB	WA	44962	44946	24767	24751	1115664	149200
1 K	Split	64	8	WB	WA	45885	45869	22808	22792	1099088	112480
1 K	Split	64	16	WB	WA	45969	45953	20667	20651	1066176	90416
8 K	Split	16	1	WT	WA	24681	24173	8283	7818	131856	66538
8 K	Split	32	1	WT	WA	15868	15612	7504	7264	186976	66538
8 K	Split	64	2	WT	WA	6590	6462	3160	3032	156000	66538
8 K	Split	16	1	WB	WNA	24681	24173	14904	6688	127304	14643
8 K	Split	32	1	WB	WNA	15868	15612	15098	6421	180200	22033
8 K	Split	64	2	WB	WNA	6590	6462	8638	2726	151104	13624

编写了 test 文件，类似于 runpublic 文件，也是依次执行 sim 指令，取自上述表格

```
#!/bin/bash

# 定义 Trace 文件路径
TRACE_FILE="ext_traces/spice.trace"

# 定义运行的命令列表
commands=(
    # --- Split Cache (Size变化) ---
    "./code/sim -bs 16 -is 8192 -ds 8192 -a 1 -wb -wa $TRACE_FILE"   # 8K Split
    "./code/sim -bs 16 -is 16384 -ds 16384 -a 1 -wb -wa $TRACE_FILE" # 16K Split
    "./code/sim -bs 16 -is 32768 -ds 32768 -a 1 -wb -wa $TRACE_FILE" # 32K Split
    "./code/sim -bs 16 -is 65536 -ds 65536 -a 1 -wb -wa $TRACE_FILE" # 64K Split

    # --- Unified Cache (Size固定, BlockSize变化) ---
    "./code/sim -bs 16 -us 8192 -a 1 -wb -wa $TRACE_FILE"            # 8K Unified BS 16
    "./code/sim -bs 32 -us 8192 -a 1 -wb -wa $TRACE_FILE"            # 8K Unified BS 32
    "./code/sim -bs 64 -us 8192 -a 1 -wb -wa $TRACE_FILE"            # 8K Unified BS 64
    "./code/sim -bs 128 -is 32768 -ds 32768 -a 1 -wb -wa $TRACE_FILE" # 32K Split BS 128

    # --- Split Cache (Associativity变化) ---
    "./code/sim -bs 64 -is 8192 -ds 8192 -a 2 -wb -wa $TRACE_FILE"   # 8K Split Assoc 2
    "./code/sim -bs 64 -is 8192 -ds 8192 -a 4 -wb -wa $TRACE_FILE"   # 8K Split Assoc 4
    "./code/sim -bs 64 -is 8192 -ds 8192 -a 8 -wb -wa $TRACE_FILE"   # 8K Split Assoc 8
    "./code/sim -bs 64 -is 8192 -ds 8192 -a 16 -wb -wa $TRACE_FILE"   # 8K Split Assoc 16
    "./code/sim -bs 64 -is 8192 -ds 8192 -a 128 -wb -wa $TRACE_FILE"   # 8K Split Assoc 128
    "./code/sim -bs 64 -is 1024 -ds 1024 -a 2 -wb -wa $TRACE_FILE"   # 1K Split Assoc 2
    "./code/sim -bs 64 -is 1024 -ds 1024 -a 8 -wb -wa $TRACE_FILE"   # 1K Split Assoc 8
    "./code/sim -bs 64 -is 1024 -ds 1024 -a 16 -wb -wa $TRACE_FILE"   # 1K Split Assoc 16

    # --- 策略变化 (WT, WNA) ---
    "./code/sim -bs 16 -is 8192 -ds 8192 -a 1 -wt -wa $TRACE_FILE"   # Write Through BS 16
    "./code/sim -bs 32 -is 8192 -ds 8192 -a 1 -wt -wa $TRACE_FILE"   # Write Through BS 32
    "./code/sim -bs 64 -is 8192 -ds 8192 -a 2 -wt -wa $TRACE_FILE"   # Write Through BS 64 Assoc 2

    "./code/sim -bs 16 -is 8192 -ds 8192 -a 1 -wa -nw $TRACE_FILE"   # Write No-Allocate
    "./code/sim -bs 32 -is 8192 -ds 8192 -a 1 -wa -nw $TRACE_FILE"   # Write No-Allocate
    "./code/sim -bs 64 -is 8192 -ds 8192 -a 2 -wa -nw $TRACE_FILE"   # Write No-Allocate
)
```

```
)

echo "-------------------------------------------------------------------------------------"
printf "%-10s | %-10s | %-10s | %-10s | %-12s | %-12s\n" "I-Miss" "I-Repl" "D-Miss" "D-Repl" "DemandFetch" "CopiesBack"
echo "-------------------------------------------------------------------------------------"

for cmd in "${commands[@]}"; do
    # 1. 运行模拟器并捕获输出
    output=$($cmd)

    # 2. 解析输出
    # 提取 Instruction Misses
    i_miss=$(echo "$output" | grep -A 5 "INSTRUCTIONS" | grep "misses" | awk '{print $2}')
    # 提取 Instruction Replacements
    i_repl=$(echo "$output" | grep -A 5 "INSTRUCTIONS" | grep "replace" | awk '{print $2}')

    # 提取 Data Misses
    d_miss=$(echo "$output" | grep -A 5 "DATA" | grep "misses" | awk '{print $2}')
    # 提取 Data Replacements
    d_repl=$(echo "$output" | grep -A 5 "DATA" | grep "replace" | awk '{print $2}')

    # 提取 Fetch 和 Copies Back
    fetch=$(echo "$output" | grep "demand fetch" | awk '{print $3}')
    cb=$(echo "$output" | grep "copies back" | awk '{print $3}')

    # 3. 打印一行数据
    printf "%-10s | %-10s | %-10s | %-10s | %-12s | %-12s\n" "$i_miss" "$i_repl" "$d_miss" "$d_repl" "$fetch" "$cb"
done
```

执行 test 函数，从终端可以看到执行所有 sim 指令后得到的结果

```
asuslinux@LAPTOP-OKJ77SU5:/mnt/e/ZJU_All/ZJU_Third/cs61c/project2/project2/attachment$ ./test.sh
-------------------------------------------------------------------------------------
I-Miss     | I-Repl     | D-Miss     | D-Repl     | DemandFetch  | CopiesBack
-------------------------------------------------------------------------------------
24681      | 24173      | 8283       | 7818       | 131856       | 12024
11514      | 10560      | 5839       | 5051       | 69412        | 8008
5922       | 4321       | 1567       | 520        | 29956        | 3628
2619       | 484        | 1290       | 103        | 15636        | 3324
36136      | 35787      | 21261      | 21098      | 229588       | 37844
26673      | 26502      | 19561      | 19476      | 369872       | 69104
23104      | 23029      | 20377      | 20324      | 695696       | 136112
1964       | 1726       | 459        | 280        | 77536        | 7296
6590       | 6462       | 3160       | 3032       | 156000       | 18880
6025       | 5897       | 875        | 747        | 110400       | 7296
6435       | 6307       | 803        | 675        | 115808       | 6656
6536       | 6408       | 799        | 671        | 117360       | 6624
6523       | 6395       | 790        | 662        | 117008       | 6576
44962      | 44946      | 24767      | 24751      | 1115664      | 149200
45885      | 45869      | 22808      | 22792      | 1099088      | 112480
45969      | 45953      | 20667      | 20651      | 1066176      | 90416
24681      | 24173      | 8283       | 7818       | 131856       | 66538
15868      | 15612      | 7504       | 7264       | 186976       | 66538
6590       | 6462       | 3160       | 3032       | 156000       | 66538
24681      | 24173      | 14904      | 6688       | 127304       | 14643
15868      | 15612      | 15098      | 6421       | 180200       | 22033
6590       | 6462       | 8638       | 2726       | 151104       | 13624
```

和 sim 中表格结果对比后，发现完全符合！设计的 Cache 是正确！！

（二）实现 trace-driven cache simulator

1．项目说明：

现在有了一个经过验证的可配置 Cache 模拟器 。使用它对提供的三个样本踪迹（spice, gcc, TeX）进行研究

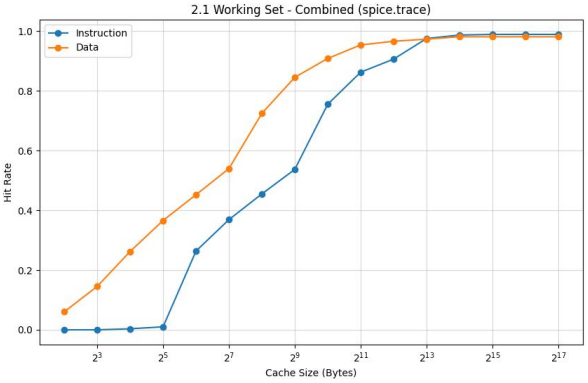
2．工作集特征 (Working Set Characterization)

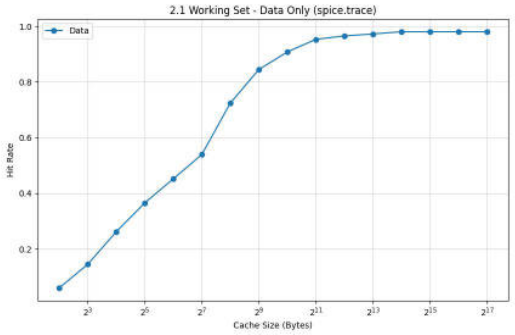
（1）实验要求：从 4 字节开始，每次增加 2 倍，直到命中率对大小不再敏感；使用分离 (Split) Cache 组织结构（分别为指令和数据绘制图表）。通过始终使用全相联 Cache 来消除冲突缺失 (conflict misses) 的影响 。设置块大小为 4 字节，使用写回 (Write-back) 和写分配 (Write-allocate) 策略。

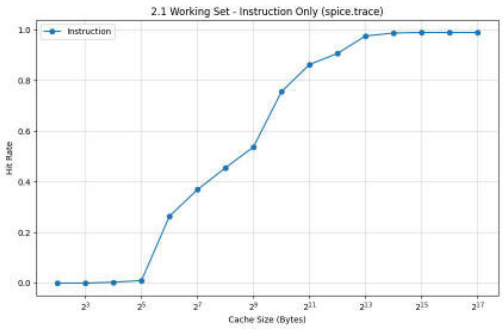
（2） 结果分析

①Spice

```
正在测试 Trace: spice.trace
------------------------------------------------
Size (B)    | Assoc     | I-HitRate     | D-HitRate
------------------------------------------------
4           | 1         | 0.0000        | 0.0604
8           | 2         | 0.0000        | 0.1454
16          | 4         | 0.0034        | 0.2622
32          | 8         | 0.0101        | 0.3659
64          | 16        | 0.2636        | 0.4521
128         | 32        | 0.3687        | 0.5395
256         | 64        | 0.4548        | 0.7243
512         | 128       | 0.5368        | 0.8448
1024        | 256       | 0.7548        | 0.9080
2048        | 512       | 0.8620        | 0.9532
4096        | 1024      | 0.9057        | 0.9656
8192        | 2048      | 0.9747        | 0.9726
16384       | 4096      | 0.9864        | 0.9806
32768       | 8192      | 0.9885        | 0.9806
65536       | 16384     | 0.9885        | 0.9806
131072      | 32768     | 0.9885        | 0.9806
```

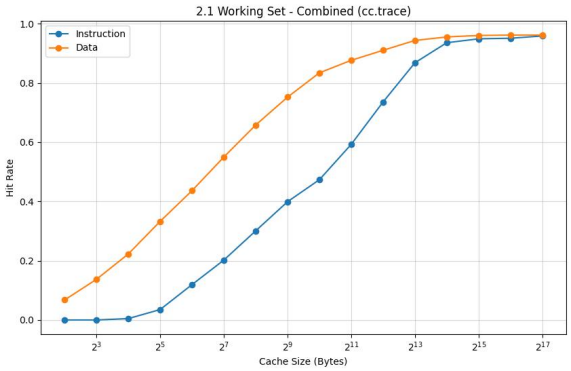


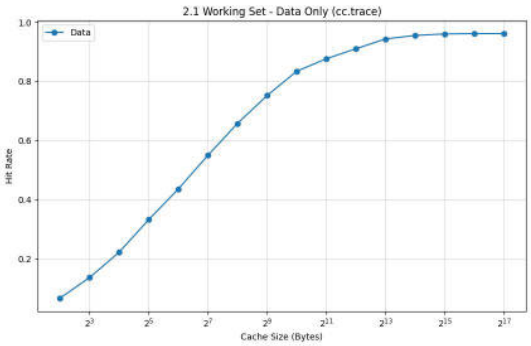


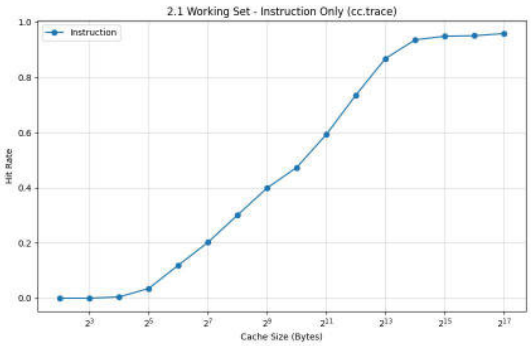


②Cc

```
正在测试 Trace: cc.trace
------------------------------------------------
Size (B)    | Assoc     | I-HitRate     | D-HitRate
------------------------------------------------
4           | 1         | 0.0000        | 0.0676
8           | 2         | 0.0000        | 0.1373
16          | 4         | 0.0047        | 0.2229
32          | 8         | 0.0353        | 0.3330
64          | 16        | 0.1194        | 0.4366
128         | 32        | 0.2023        | 0.5502
256         | 64        | 0.3010        | 0.6583
512         | 128       | 0.3993        | 0.7523
1024        | 256       | 0.4736        | 0.8344
2048        | 512       | 0.5930        | 0.8766
4096        | 1024      | 0.7360        | 0.9105
8192        | 2048      | 0.8684        | 0.9437
16384       | 4096      | 0.9360        | 0.9555
32768       | 8192      | 0.9492        | 0.9606
65536       | 16384     | 0.9511        | 0.9618
131072      | 32768     | 0.9588        | 0.9618
 -> 图表已保存: plots/task2_1_cc.trace_combined.png
 -> 图表已保存: plots/task2_1_cc.trace_instr.png
 -> 图表已保存: plots/task2_1_cc.trace_data.png
```

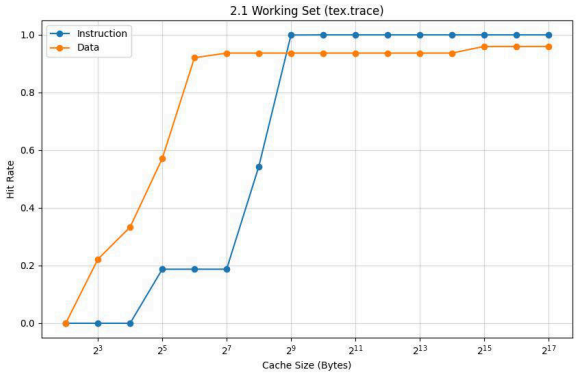


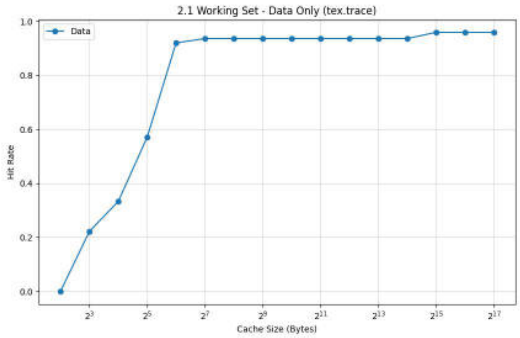


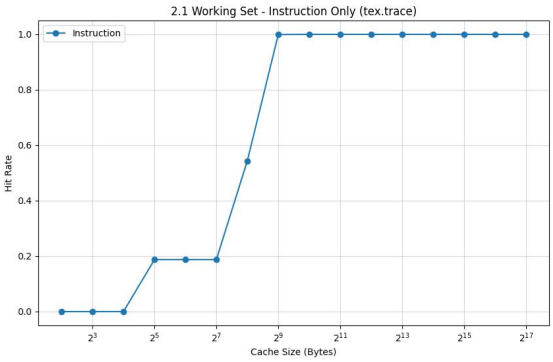


③Tex

```
正在测试 Trace: tex.trace
------------------------------------------------
Size (B)   | Assoc    | I-HitRate     | D-HitRate
------------------------------------------------
4          | 1        | 0.0000        | 0.0001
8          | 2        | 0.0000        | 0.2222
16         | 4        | 0.0000        | 0.3333
32         | 8        | 0.1874        | 0.5713
64         | 16       | 0.1874        | 0.9204
128        | 32       | 0.1874        | 0.9365
256        | 64       | 0.5433        | 0.9365
512        | 128      | 0.9991        | 0.9365
1024       | 256      | 0.9997        | 0.9365
2048       | 512      | 0.9997        | 0.9365
4096       | 1024     | 0.9997        | 0.9365
8192       | 2048     | 0.9997        | 0.9365
16384      | 4096     | 0.9997        | 0.9365
32768      | 8192     | 0.9997        | 0.9595
65536      | 16384    | 0.9997        | 0.9595
131072     | 32768    | 0.9997        | 0.9595
 -> 图表已保存: plots/task2_1_tex.trace_combined.png
 -> 图表已保存: plots/task2_1_tex.trace_instr.png
 -> 图表已保存: plots/task2_1_tex.trace_data.png
```







（3） 回答问题

①解释这个实验在做什么，它是如何工作的。并解释命中率与 Cache 大小关系图中特征的意义

1）这个实验旨在通过模拟，找出程序在运行过程中频繁访问的“核心数据量”和“核心指令量”，即工作集大小

2）工作原理：通过设置全相联，彻底消除了冲突缺失。因为全相联 Cache 只要有空位就能放数据，不存在“运气不好映到同一行”的情况。块大小固定为极小的 4 字节，这是为了尽

量减少空间局部性带来的预取效应。从很小的 Cache 开始，此时 Cache 一定装不下程序所需的数据，会发生大量的容量缺失。随着 Cache 容量不断翻倍，能装下的数据越来越多。当 Cache 大小增加到某一个值时，命中率不再显著上升，说明此时的 Cache 已经大到足以装下程序当前阶段频繁使用的所有数据了。这个“转折点”对应的大小，就是工作集大小

3）意义：

a．快速上升阶段：此时 Cache 小于工作集大小。由于容量不足，频繁发生容量缺失，数据刚进来就被踢出去。每增加一点容量，就能显著减少这种颠簸，因此命中率快速上升。

b．拐点：曲线斜率发生急剧变化的地方。这代表 Cache 大小刚刚好能容纳下程序的主要工作集。

c．平坦阶段：此时 Cache 大于工作集大小。主要的缺失是由于第一次访问数据，无论 Cache 多大都无法避免的。继续增加容量对命中率几乎没有贡献。

②这三个样本踪迹的总指令工作集大小和数据工作集大小分别是多少

1）spice.trace 的数据工作集大小约为 2^14Bytes = 16KB ，指令工作集大小约为 2^14Bytes = 16KB

2）cc.trace 的数据工作集大小约为 2^14Bytes = 16KB ， 指令工作集大小约为 2^14Bytes = 16KB

3）tex.trace 的数据工作集大小约为 2^6Bytes = 128B ， 指令工作集大小也约为 2^9Bytes = 512B

3．块大小的影响 (Impact of Block Size)

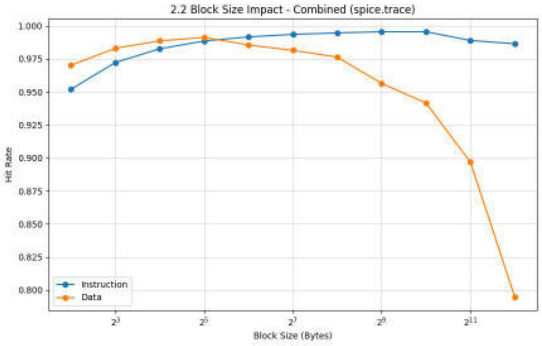
（1）实验要求：设置：分离 I- D-Cache，各 8 KB。关联度为 2，写回，写分配。绘制 Cache 命中率随块大小变化的函数图。块大小在 4 字节到 4 KB 之间变化（2 的幂）。对三个踪迹分别绘制指令和数据的图

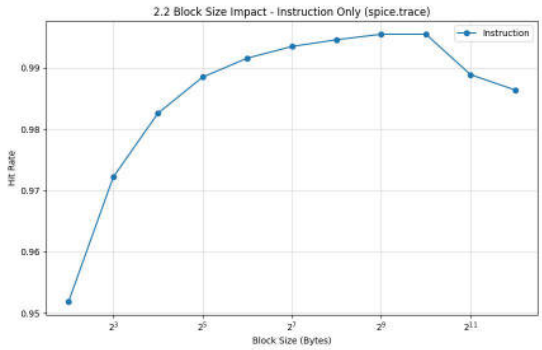
（2）结果分析

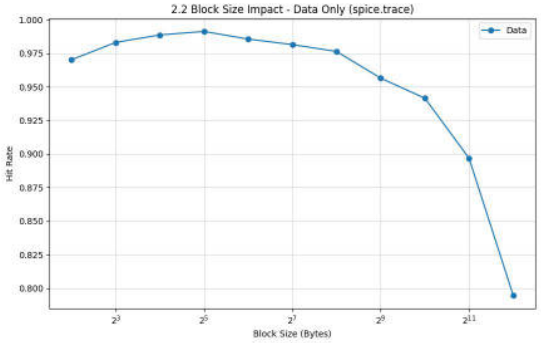
①Spice

```
Running Task 2.2: Impact of Block Size

正在测试 Trace: spice.trace
--------------------------------------------------
BlockSize  | I-HitRate     | D-HitRate
--------------------------------------------------
4          | 0.9519        | 0.9701
8          | 0.9722        | 0.9830
16         | 0.9826        | 0.9886
32         | 0.9885        | 0.9912
64         | 0.9916        | 0.9855
128        | 0.9935        | 0.9814
256        | 0.9946        | 0.9763
512        | 0.9955        | 0.9564
1024       | 0.9955        | 0.9415
2048       | 0.9889        | 0.8968
4096       | 0.9864        | 0.7947
 -> 图表已保存: plots/task2_2_spice.trace.png
```

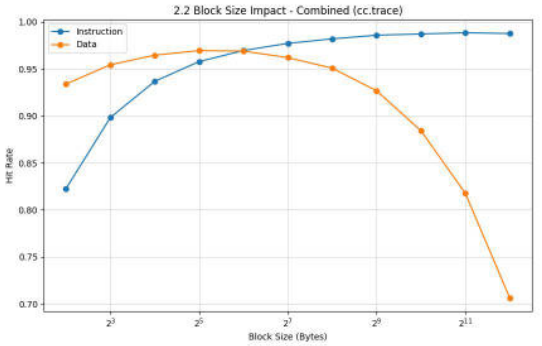


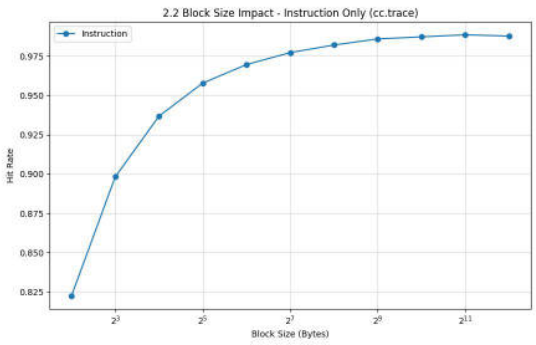


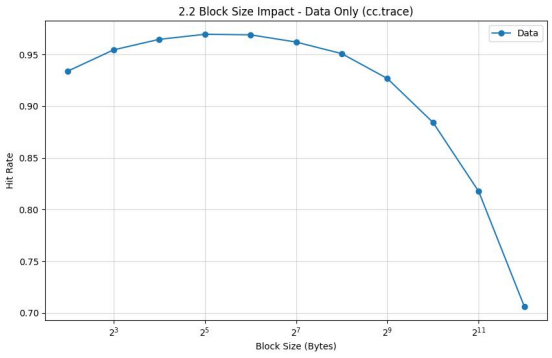


②Cc

```
正在测试 Trace: cc.trace
--------------------------------------------------
BlockSize  | I-HitRate     | D-HitRate
--------------------------------------------------
4          | 0.8225        | 0.9339
8          | 0.8982        | 0.9544
16         | 0.9368        | 0.9646
32         | 0.9578        | 0.9695
64         | 0.9696        | 0.9690
128        | 0.9772        | 0.9620
256        | 0.9820        | 0.9508
512        | 0.9859        | 0.9267
1024       | 0.9872        | 0.8842
2048       | 0.9885        | 0.8175
4096       | 0.9877        | 0.7062
 -> 图表已保存: plots/task2_2_cc.trace_combined.png
 -> 图表已保存: plots/task2_2_cc.trace_instr.png
 -> 图表已保存: plots/task2_2_cc.trace_data.png
```

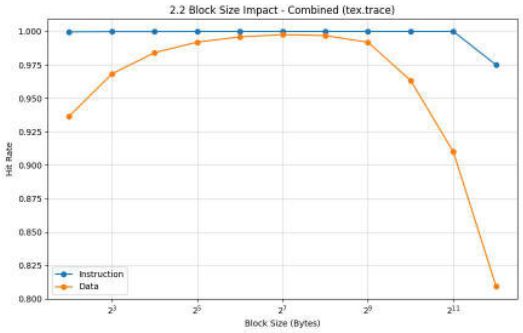


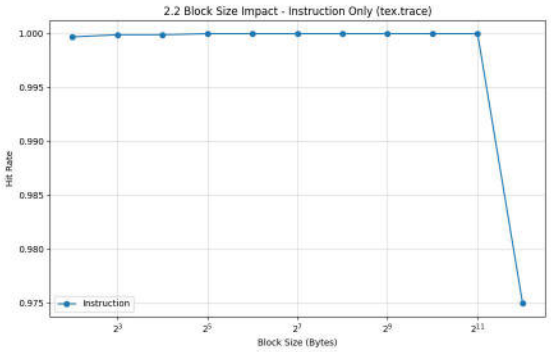


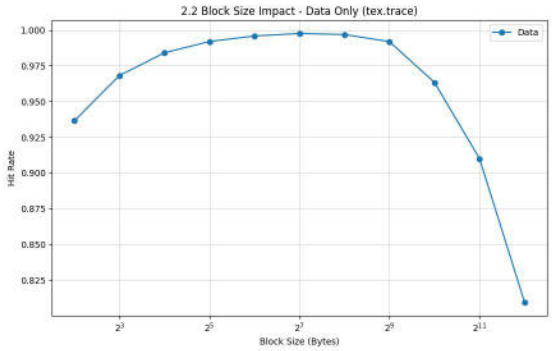


③Tex

```
正在测试 Trace: tex.trace
--------------------------------------------------
BlockSize  | I-HitRate     | D-HitRate
--------------------------------------------------
4          | 0.9997        | 0.9365
8          | 0.9999        | 0.9682
16         | 0.9999        | 0.9841
32         | 1.0000        | 0.9920
64         | 1.0000        | 0.9959
128        | 1.0000        | 0.9976
256        | 1.0000        | 0.9969
512        | 1.0000        | 0.9919
1024       | 1.0000        | 0.9633
2048       | 1.0000        | 0.9100
4096       | 0.9750        | 0.8095
 -> 图表已保存: plots/task2_2_tex.trace_combined.png
 -> 图表已保存: plots/task2_2_tex.trace_instr.png
 -> 图表已保存: plots/task2_2_tex.trace_data.png
```







（3）问题解答

①解释为什么命中率与块大小的关系图会呈现这种形状。特别是解释空间局部性 (spatial locality) 与曲线形状的相关性。

1）上升阶段（左侧）：随着块大小从 4B 增加到 32B/64B，命中率迅速上升，原因是：当块大小较小（如 4B）时，Cache 每次只把当前需要的一个字搬进来。如果程序接下来要读下一个字（顺序执行指令或遍历数组），Cache 里没有，必须再次访问内存，导致 Miss。随着块变大，Cache 利用了空间局部性，在一次缺失（Miss）时，顺便把后面紧邻的数据（预取）都搬进来了。下次访问邻近数据时就是 Hit。因此，块越大，预取效果越好，命中率上升

2）峰值（中间）：在某个特定大小达到最高点，然后趋于平稳，在这个阶段，缓存块大小已经足够大，能够覆盖程序的大部分空间局部性需求。即使块大小继续增加，对命中率的提升也不再显著

3）下降阶段（右侧）：当块大小继续增大，命中率反而开始下降，甚至急剧恶化，原因是：Cache 污染：块太大了，导致搬进来了很多程序根本用不到的“垃圾数据”。这些没用的数据占据了宝贵的 Cache 空间，挤走了有用的数据；冲突缺失增加，因为 cache 总大小是固定的，当块大小变大时，Cache 中的 Blocks 会急剧减少。这会导致不同地址映射到同一行的概率激增，从而引发严重的冲突缺失。

4）不同的工作集总体曲线特征符合上述描述，但是曲线的形状和拐点位置不同，说明不同程序的空间局部性不同，对块大小的需求也不同，所以导致了不同的曲线形状。

②每个踪迹的最佳块大小是多少（分别考虑指令和数据）？

1）Spice：指令：当块大小达到 1024B 时，命中率接近最高，之后随着块大小的增加，命中率略有下降。数据：数据命中率在块大小为 32B 时达到最高，之后随着块大小的增加，命中率开始下降。

2）Cc：指令：当块大小达到 2048B 时，命中率接近最高，之后随着块大小的增加，命中率略有下降。数据：数据命中率在块大小为 32B 时达到最高，之后随着块大小的增加，命中率开始下降。

3）Tex：指令：当块大小在 4B~2048B 时，命中率接近最高，保持稳定，之后随着块大小的增加，命中率略有下降。数据：数据命中率在块大小为 128B 时达到最高，之后随着块大小的增加，命中率开始下降。

③指令和数据的最佳块大小是否不同？这说明了指令引用与数据引用的性质有何不同？

1）通常是不同的。 甚至即使最佳点数值相同，曲线的走势通常也不同。

2）指令流的曲线通常比较平缓，且最佳块大小往往较大。指令引用的性质：极高的空间局部性：程序通常是顺序执行的（PC = PC + 4），除非遇到跳转分支；顺序性强：这意味着“预取”非常有效。加载一个大块通常是划算的，因为 CPU 很有可能把它们都执行一遍；只读：指令不会被修改，不会因为写操作导致复杂的脏块替换问题

3）数据流的曲线通常比较陡峭，达到峰值后，如果块继续变大，命中率下降得更快。数据引用的性质：局部性较弱/复杂：虽然数组访问有空间局部性，但很多程序使用链表、树、哈希表或指针，这些数据在内存中是跳跃分布的；容易污染：如果数据访问是随机跳跃的，加载一个大块进来，可能只用到了开头的 4 个字节，剩下的 252 字节都是浪费的“污染”，不仅没用，还占了位置。

4．关联度的影响 (Impact of Associativity)

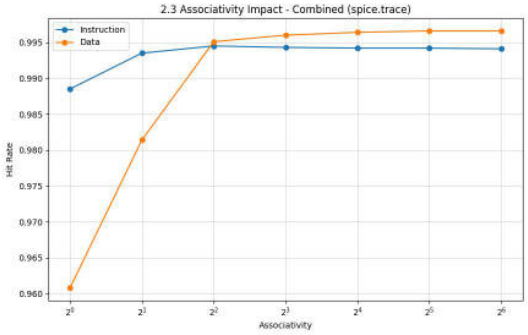
（1）实验要求：设置：分离 I- D-Cache，各 8 KB。块大小 128 字节，写回，写分配。绘制 Cache 命中率随关联度变化的函数图。关联度在 1 到 64 之间变化（2 的幂）。对三个踪迹分别绘制指令和数据的图

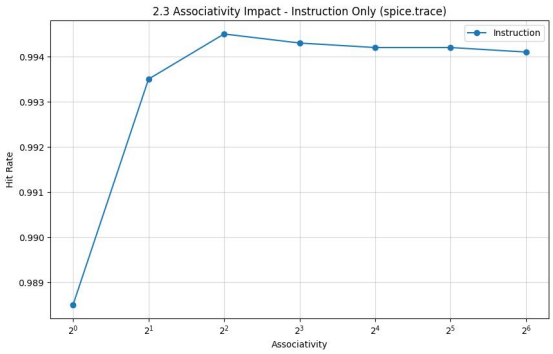
（2）结果分析

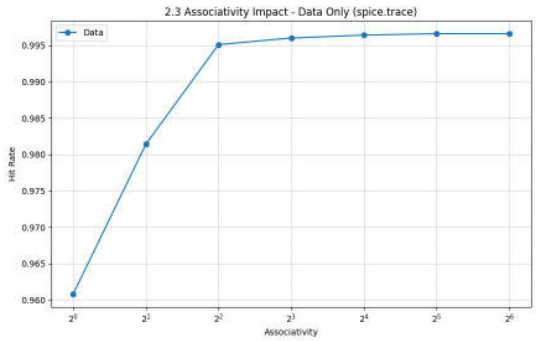
①Spice

```
Running Task 2.3: Impact of Associativity

正在测试 Trace: spice.trace
-------------------------------------------------
Assoc      | I-HitRate     | D-HitRate
-------------------------------------------------
1          | 0.9885        | 0.9608
2          | 0.9935        | 0.9814
4          | 0.9945        | 0.9951
8          | 0.9943        | 0.9960
16         | 0.9942        | 0.9964
32         | 0.9942        | 0.9966
64         | 0.9941        | 0.9966
 -> 图表已保存: plots/task2_3_spice.trace_combined.png
 -> 图表已保存: plots/task2_3_spice.trace_instr.png
 -> 图表已保存: plots/task2_3_spice.trace_data.png
```

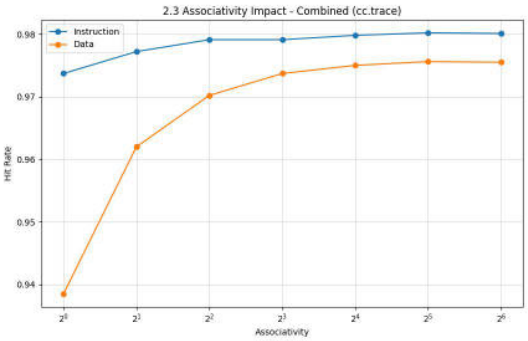


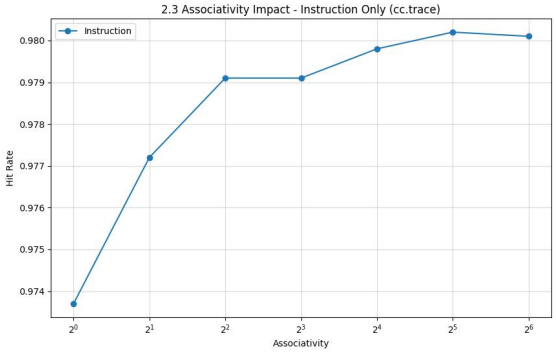


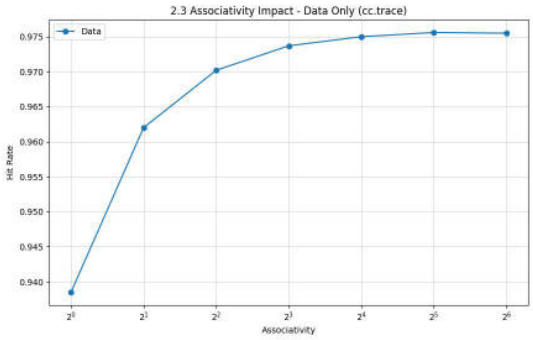


②Cc

```
正在测试 Trace: cc.trace
-------------------------------------------------
Assoc      | I-HitRate     | D-HitRate
-------------------------------------------------
1          | 0.9737        | 0.9385
2          | 0.9772        | 0.9620
4          | 0.9791        | 0.9702
8          | 0.9791        | 0.9737
16         | 0.9798        | 0.9750
32         | 0.9802        | 0.9756
64         | 0.9801        | 0.9755
 -> 图表已保存: plots/task2_3_cc.trace_combined.png
 -> 图表已保存: plots/task2_3_cc.trace_instr.png
 -> 图表已保存: plots/task2_3_cc.trace_data.png
```

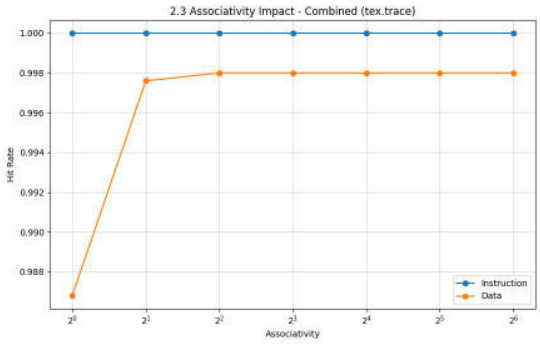


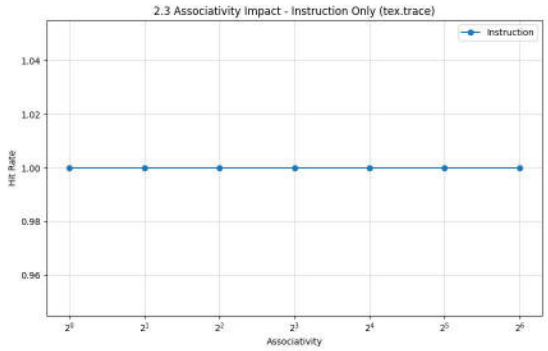


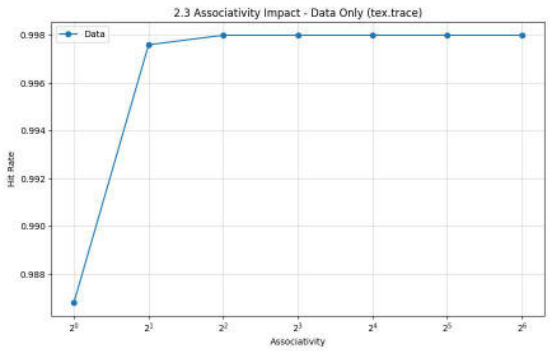


③Tex

```
正在测试 Trace: tex.trace
--------------------------------------------------
Assoc      | I-HitRate     | D-HitRate
--------------------------------------------------
1          | 1.0000        | 0.9868
2          | 1.0000        | 0.9976
4          | 1.0000        | 0.9980
8          | 1.0000        | 0.9980
16         | 1.0000        | 0.9980
32         | 1.0000        | 0.9980
64         | 1.0000        | 0.9980
  -> 图表已保存: plots/task2_3_tex.trace_combined.png
  -> 图表已保存: plots/task2_3_tex.trace_instr.png
  -> 图表已保存: plots/task2_3_tex.trace_data.png
asuslinux@LAPTOP-OKJ77SU5:/mnt/e/ZJU_All/ZJU_Third/cs61c/project2/project2/attachment$
```







（3）问题解答

①解释为什么命中率与关联度的关系图会呈现这种形状。

1）曲线呈现典型的“边际收益递减”特征，初期：命中率急剧上升；中期：上升幅度变缓；后期：曲线几乎变平，命中率几乎不再随关联度增加而变化。

2）直接映射 (Associativity = 1) 的局限：在直接映射 Cache 中，每个内存块只能放在 Cache 中的唯一一个位置。如果程序需要交替访问两个映射到同一个位置的地址，它们会互相“踢出”对方，导致严重的冲突缺失

3）增加关联度：当关联度从 1 增加到 2 或 4 时，意味着每个 Set 里可以容纳 2 个或 4 个块。A 和 B 可以和平共处在同一个 Set 里，冲突缺失被大量消除，因此命中率显著提升。

4）当关联度增加到一定程度（比如 4 路或 8 路）时，绝大多数由“运气不好”导致的冲突缺失已经被消除了。此时剩下的缺失主要是容量缺失（因为 Cache 固定是 8KB，如果工作集大于 8KB，关联度再高也没用）和 强制缺失。关联度无法解决容量不足和第一次访问的问题，所以曲线最终会趋于平坦。

5）下降问题：当关联度继续增加超过某个阈值后，命中率可能会略有下降。可能是由于缓存结构的复杂性增加，导致缓存管理的开销增大

②指令和数据引用的图表有区别吗？这说明关联度对指令与数据引用的影响有何不同？

1）通常是有区别的

2）指令 (Instruction)：曲线通常比较平缓，或者在关联度很低时就迅速达到饱和。在直接映射下，指令的命中率通常也不会太差。指令引用的性质：顺序性强：指令通常是按地址顺序执行的，具有很好的空间局部性。不易冲突：代码段通常是连续存放的，除非程序在两个距离恰好是 Cache Size 倍数的函数之间频繁跳转，否则很少发生严重的冲突。因此指令 Cache 对关联度不敏感。通常 2 路组相联就足够让它达到很好的性能了，再高只是浪费硬件资源。

3）数据 (Data)：在直接映射下，命中率可能非常低。当关联度增加时，数据的命中率提升幅度通常比指令更陡峭、更显著。数据引用的性质：随机性强：数据在内存中的分布非常杂乱。栈、堆、全局变量分布在内存的不同区域。容易冲突：程序经常需要同时访问栈上的局部变量、堆上的动态数组和全局区的数据。这些不同区域的地址很容易“运气不好”地映射到同一个 Cache Set 中。因此，数据 Cache 对关联度非常敏感。在低关联度下，数据很容易发生颠簸；增加关联度能“救活”这些因地址冲突而被频繁踢出的数据，从而带来巨大的性能提升。

5．内存带宽 (Memory Bandwidth) 比较写直达 (Write-through) 与写回 (Write-back) Cache 产生的总内存流量

（1）实验要求：使用分离 I- D-Cache。模拟几种合理的 Cache 大小（如 8KB, 16KB）、块大小（如 64B, 128B）和关联度（如 2, 4）

（2）实验 1：固定使用写不分配 (Write-no-allocate) 策略。运行 4 或 5 次不同的模拟

①结果分析：

1）Spice

```
>>> 正在分析 Trace: spice.trace <<<

[实验 1] Write-Through (WT) vs Write-Back (WB) | Policy: Write-No-Allocate
-------------------------------------------------------------------------------------------------
bs      | is      | ds      | a     | Write Pol     | Alloc Pol         | Fetch         | Copies
-------------------------------------------------------------------------------------------------
64      | 8192    | 8192    | 2     | WT            | No-Alloc          | 151104        | 66538
64      | 8192    | 8192    | 2     | WB            | No-Alloc          | 151104        | 13624
64      | 8192    | 8192    | 4     | WT            | No-Alloc          | 107296        | 66538
64      | 8192    | 8192    | 4     | WB            | No-Alloc          | 107296        | 9219
128     | 8192    | 8192    | 2     | WT            | No-Alloc          | 280768        | 66538
128     | 8192    | 8192    | 2     | WB            | No-Alloc          | 280768        | 32287
128     | 8192    | 8192    | 4     | WT            | No-Alloc          | 162240        | 66538
128     | 8192    | 8192    | 4     | WB            | No-Alloc          | 162240        | 13733
64      | 16384   | 16384   | 2     | WT            | No-Alloc          | 57008         | 66538
64      | 16384   | 16384   | 2     | WB            | No-Alloc          | 57008         | 8252
64      | 16384   | 16384   | 4     | WT            | No-Alloc          | 38064         | 66538
64      | 16384   | 16384   | 4     | WB            | No-Alloc          | 38064         | 7681
128     | 16384   | 16384   | 2     | WT            | No-Alloc          | 93632         | 66538
128     | 16384   | 16384   | 2     | WB            | No-Alloc          | 93632         | 9880
128     | 16384   | 16384   | 4     | WT            | No-Alloc          | 64320         | 66538
128     | 16384   | 16384   | 4     | WB            | No-Alloc          | 64320         | 9668
```

2）Cc

```
>>> 正在分析 Trace: cc.trace <<<

[实验 1] Write-Through (WT) vs Write-Back (WB) | Policy: Write-No-Allocate
-------------------------------------------------------------------------------------------------
bs      | is      | ds      | a     | Write Pol     | Alloc Pol         | Fetch         | Copies
-------------------------------------------------------------------------------------------------
64      | 8192    | 8192    | 2     | WT            | No-Alloc          | 474256        | 83030
64      | 8192    | 8192    | 2     | WB            | No-Alloc          | 474256        | 39426
64      | 8192    | 8192    | 4     | WT            | No-Alloc          | 423104        | 83030
64      | 8192    | 8192    | 4     | WB            | No-Alloc          | 423104        | 32102
128     | 8192    | 8192    | 2     | WT            | No-Alloc          | 824192        | 83030
128     | 8192    | 8192    | 2     | WB            | No-Alloc          | 824192        | 78872
128     | 8192    | 8192    | 4     | WT            | No-Alloc          | 716736        | 83030
128     | 8192    | 8192    | 4     | WB            | No-Alloc          | 716736        | 59292
64      | 16384   | 16384   | 2     | WT            | No-Alloc          | 252096        | 83030
64      | 16384   | 16384   | 2     | WB            | No-Alloc          | 252096        | 24048
64      | 16384   | 16384   | 4     | WT            | No-Alloc          | 205120        | 83030
64      | 16384   | 16384   | 4     | WB            | No-Alloc          | 205120        | 19588
128     | 16384   | 16384   | 2     | WT            | No-Alloc          | 440768        | 83030
128     | 16384   | 16384   | 2     | WB            | No-Alloc          | 440768        | 40538
128     | 16384   | 16384   | 4     | WT            | No-Alloc          | 353632        | 83030
128     | 16384   | 16384   | 4     | WB            | No-Alloc          | 353632        | 31103
```

3）Tex

```
>>> 正在分析 Trace: tex.trace <<<

[实验 1] Write-Through (WT) vs Write-Back (WB) | Policy: Write-No-Allocate
-----------------------------------------------------------------------------------------------
bs     | is     | ds     | a    | Write Pol    | Alloc Pol        | Fetch        | Copies
-----------------------------------------------------------------------------------------------
64     | 8192   | 8192   | 2    | WT           | No-Alloc         | 2880         | 104513
64     | 8192   | 8192   | 2    | WB           | No-Alloc         | 2880         | 29903
64     | 8192   | 8192   | 4    | WT           | No-Alloc         | 3312         | 104513
64     | 8192   | 8192   | 4    | WB           | No-Alloc         | 3312         | 29903
128    | 8192   | 8192   | 2    | WT           | No-Alloc         | 3488         | 104513
128    | 8192   | 8192   | 2    | WB           | No-Alloc         | 3488         | 29935
128    | 8192   | 8192   | 4    | WT           | No-Alloc         | 4064         | 104513
128    | 8192   | 8192   | 4    | WB           | No-Alloc         | 4064         | 29935
64     | 16384  | 16384  | 2    | WT           | No-Alloc         | 2464         | 104513
64     | 16384  | 16384  | 2    | WB           | No-Alloc         | 2464         | 29909
64     | 16384  | 16384  | 4    | WT           | No-Alloc         | 2464         | 104513
64     | 16384  | 16384  | 4    | WB           | No-Alloc         | 2464         | 29909
128    | 16384  | 16384  | 2    | WT           | No-Alloc         | 2656         | 104513
128    | 16384  | 16384  | 2    | WB           | No-Alloc         | 2656         | 29957
128    | 16384  | 16384  | 4    | WT           | No-Alloc         | 2656         | 104513
128    | 16384  | 16384  | 4    | WB           | No-Alloc         | 2656         | 29957
```

②问题回答：

1）哪种 Cache 的内存流量更小，写直达还是写回？为什么？

a．通常情况下，写回 (Write-Back) 的内存流量要小得多

b．Write-Back 利用了“时间局部性”。如果程序对同一个内存块进行了 100 次写操作，Write-Back 只是在 Cache 中修改 100 次（标记为脏），只有当这个块最终被替换出去时，才产生 1 次 写内存的流量。Write-Through 策略下，每一次 CPU 的写操作（Store 指令）都会迫使数据立即发送到总线写入内存。100 次写操作就会产生 100 次 内存流量。

2）是否有任何情况下上述问题的答案会反转？请解释 。

a．这种情况通常发生在 “写稀疏” 且 “块大小很大” 的场景下，假设 Cache 块大小为 64 字节。程序只修改了块中的 1 个字，然后这个块就被替代了。Write-Through 流量：只写了 1 个字到内存。Write-Back 流量：虽然只改了 1 个字，但因为块变脏了，驱逐时必须把整个 64 字节的块写回内存。

b．如果缓存替换非常频繁，回写 Write-Back 缓存可能会导致大量的写回操作，从而增加内存流量。而通写 Write-Through 缓存每次写操作都立即写回主存，避免了替换时的写回操作

（3）实验 2：固定使用写回 (Write-back) 策略。进行相同的实验，但这次比较写分配 (Write-allocate) 和写不分配 (Write-no-allocate) 策略

①结果分析

1）Spice

```
[实验 2] Write-Allocate (WA) vs No-Allocate (WNA) | Policy: Write-Back
-------------------------------------------------------------------------------------------
bs     | is     | ds     | a    | Write Pol    | Alloc Pol        | Fetch        | Copies
-------------------------------------------------------------------------------------------
64     | 8192   | 8192   | 2    | WB           | Allocate         | 156000       | 18880
64     | 8192   | 8192   | 2    | WB           | No-Alloc         | 151104       | 13624
64     | 8192   | 8192   | 4    | WB           | Allocate         | 110400       | 7296
64     | 8192   | 8192   | 4    | WB           | No-Alloc         | 107296       | 9219
128    | 8192   | 8192   | 2    | WB           | Allocate         | 291584       | 36256
128    | 8192   | 8192   | 2    | WB           | No-Alloc         | 280768       | 32287
128    | 8192   | 8192   | 4    | WB           | Allocate         | 171136       | 14592
128    | 8192   | 8192   | 4    | WB           | No-Alloc         | 162240       | 13733
64     | 16384  | 16384  | 2    | WB           | Allocate         | 59856        | 6208
64     | 16384  | 16384  | 2    | WB           | No-Alloc         | 57008        | 8252
64     | 16384  | 16384  | 4    | WB           | Allocate         | 39728        | 5280
64     | 16384  | 16384  | 4    | WB           | No-Alloc         | 38064        | 7681
128    | 16384  | 16384  | 2    | WB           | Allocate         | 99008        | 9088
128    | 16384  | 16384  | 2    | WB           | No-Alloc         | 93632        | 9880
128    | 16384  | 16384  | 4    | WB           | Allocate         | 66304        | 7200
128    | 16384  | 16384  | 4    | WB           | No-Alloc         | 64320        | 9668
```

2）Cc

```
[实验 2] Write-Allocate (WA) vs No-Allocate (WNA) | Policy: Write-Back
-------------------------------------------------------------------------------------------
bs     | is     | ds     | a    | Write Pol    | Alloc Pol        | Fetch        | Copies
-------------------------------------------------------------------------------------------
64     | 8192   | 8192   | 2    | WB           | Allocate         | 488976       | 41840
64     | 8192   | 8192   | 2    | WB           | No-Alloc         | 474256       | 39426
64     | 8192   | 8192   | 4    | WB           | Allocate         | 435408       | 33216
64     | 8192   | 8192   | 4    | WB           | No-Alloc         | 423104       | 32102
128    | 8192   | 8192   | 2    | WB           | Allocate         | 848832       | 91744
128    | 8192   | 8192   | 2    | WB           | No-Alloc         | 824192       | 78872
128    | 8192   | 8192   | 4    | WB           | Allocate         | 737632       | 68384
128    | 8192   | 8192   | 4    | WB           | No-Alloc         | 716736       | 59292
64     | 16384  | 16384  | 2    | WB           | Allocate         | 261248       | 23584
64     | 16384  | 16384  | 2    | WB           | No-Alloc         | 252096       | 24048
64     | 16384  | 16384  | 4    | WB           | Allocate         | 212176       | 18208
64     | 16384  | 16384  | 4    | WB           | No-Alloc         | 205120       | 19588
128    | 16384  | 16384  | 2    | WB           | Allocate         | 455840       | 46048
128    | 16384  | 16384  | 2    | WB           | No-Alloc         | 440768       | 40538
128    | 16384  | 16384  | 4    | WB           | Allocate         | 363392       | 32512
128    | 16384  | 16384  | 4    | WB           | No-Alloc         | 353632       | 31103
```

3）Tex

```
[实验 2] Write-Allocate (WA) vs No-Allocate (WNA) | Policy: Write-Back
-------------------------------------------------------------------------------------------------
bs     | is     | ds     | a    | Write Pol    | Alloc Pol        | Fetch        | Copies
-------------------------------------------------------------------------------------------------
64     | 8192   | 8192   | 2    | WB           | Allocate         | 15696        | 7664
64     | 8192   | 8192   | 2    | WB           | No-Alloc         | 2880         | 29903
64     | 8192   | 8192   | 4    | WB           | Allocate         | 15408        | 7568
64     | 8192   | 8192   | 4    | WB           | No-Alloc         | 3312         | 29903
128    | 8192   | 8192   | 2    | WB           | Allocate         | 18528        | 8608
128    | 8192   | 8192   | 2    | WB           | No-Alloc         | 3488         | 29935
128    | 8192   | 8192   | 4    | WB           | Allocate         | 15648        | 7648
128    | 8192   | 8192   | 4    | WB           | No-Alloc         | 4064         | 29935
64     | 16384  | 16384  | 2    | WB           | Allocate         | 11696        | 7616
64     | 16384  | 16384  | 2    | WB           | No-Alloc         | 2464         | 29909
64     | 16384  | 16384  | 4    | WB           | Allocate         | 10480        | 7568
64     | 16384  | 16384  | 4    | WB           | No-Alloc         | 2464         | 29909
128    | 16384  | 16384  | 2    | WB           | Allocate         | 13696        | 8256
128    | 16384  | 16384  | 2    | WB           | No-Alloc         | 2656         | 29957
128    | 16384  | 16384  | 4    | WB           | Allocate         | 11200        | 7648
128    | 16384  | 16384  | 4    | WB           | No-Alloc         | 2656         | 29957
```

②问题回答：

1）哪种 Cache 的内存流量更小，写分配还是写不分配？为什么？

a．通常情况下，写不分配的内存流量更小，对于 spice.trace 和 tex.spice 测试结果，写不分配缓存具有较小的内存流量；对于 tex.trace 测试结果，写分配缓存具有较小的内存流量。

b．Write-Allocate 当发生写缺失时，Write-Allocate 必须先把这块数据从内存读入 (Fetch) Cache，然后再修改。Write-No-Allocate 当发生写缺失时，它直接把要写的那部分数据写入内存，不进行读取，不会将新的缓存行加载到缓存中，这可以减少从主存取数据的次数。通常“读入整个块”的流量远大于“直接写一个字”的流量

2）是否有任何情况下上述问题的答案会反转？请解释 。

a．这种情况发生在 “写缺失后紧接着读取同一块数据”的场景，程序对地址 A 进行写操作（Miss），紧接着又读取地址 A 或其邻近地址

b．对于写不分配，写 Miss 直接写内存（产生写流量），读 Miss 因为刚才没把块读进来，现在读必须去内存取整个块（产生读流量）；对于写分配，写 Miss 取整个块（产生读流量），读 Hit 因为刚才已经把块读进来了，这次直接命中（无流量），Write-Allocate 通过一次读取解决了两次访问需求，总流量反而比 Write-No-Allocate 小

c．在程序的空间局部性较高，写操作频繁且数据经常被重写的情况下，写分配缓存可能会产生较小的内存流量。因为写分配缓存会在写操作时加载新的缓存行，并在之后的写操作中重复使用这些缓存行，从而减少了从主存取数据的次数，减少了内存流量

三、总结与感悟

1．实验工作总结

本次实验中，我设计并实现了一个功能完备的踪迹驱动（Trace-Driven）Cache 模拟器。该模拟器支持高度可配置的参数，包括缓存总大小、块大小、关联度、分离/统一结构、写策略以及写分配策略。在开发过程中，我采用了增量式开发的策略：

首先实现了基础的直接映射（Direct-Mapped）和统一缓存（Unified Cache）。接着引入了双向链表结构来维护 LRU（最近最少使用）替换算法，实现了全相联和组相联功能。随后扩展了分离式缓存（Split Cache）的逻辑，并完善了写策略和写分配策略的复杂的脏位（Dirty Bit）处理逻辑。最后，通过 diff 命令与提供的标准输出文件进行了对比验证，并通过了所有公开测试集，确保了模拟器的正确性。

2．实验结果分析与核心发现

在性能评估阶段，我编写了 Python 和 Shell 脚本实现了自动化测试与数据可视化，对三个典型的踪迹文件（spice, cc, tex）进行了深入分析，得出了以下关键结论：

工作集特征（Task 2.1）：通过观察命中率随 Cache 大小变化的拐点，我直观地理解了“工作集”的概念。当 Cache 容量覆盖主要工作集时，命中率会显著提升；之后再增加容量，收益仅限于减少强制缺失，提升幅度变缓。

块大小的权衡（Task 2.2）：实验验证了块大小与空间局部性的关系。块太小无法利用空间局部性；块太大则会导致“缓存污染”并减少行数从而引发冲突缺失。我发现对于大多数测试集，32B 到 128B 往往是最佳平衡点。

关联度的收益（Task 2.3）：关联度的增加能显著消除冲突缺失，尤其是对于访问模式较随机的数据（Data）引用，收益远大于指令（Instruction）引用。但也验证了边际收益递减规律，通常 4 路或 8 路组相联已能达到很好的性能。

内存带宽分析（Task 2.4）：通过对比不同写策略，我发现 Write-Back 通常比 Write-Through 节省大量带宽，但在“写稀疏”且块很大的极端场景下可能反转；Write-No-Allocate 通常比 Write-Allocate 流量更小，除非程序具有极高的“写后读”局部性。

3．遇到的困难与解决

LRU 链表维护：在处理替换逻辑时，双向链表的 insert 和 delete 操作容易出现断链。我通过画图模拟节点指针的变换，确立了正确的链表操作顺序，并特别处理了链表为空或仅

有一个节点时的边界情况。

自动化数据处理：手动运行几十次模拟非常低效。我学习并编写了 Python 脚本，利用 subprocess 调用模拟器并用正则表达式（Regular Expression）提取日志中的命中率和流量数据，自动生成图表，极大地提高了实验效率。

4．心得体会

通过本次实验，我不再仅仅停留在课本上对 Cache "Tag, Index, Offset" 的抽象理解，而是通过每一行 C 代码的编写，深刻体会了 Cache 地址映射、块替换和写回机制的具体实现细节。 特别是通过亲手绘制性能曲线，我理解了计算机体系结构设计中充满了“权衡（Trade-off）”的艺术——没有绝对完美的参数（如块大小不是越大越好），只有最适合特定负载的配置。此外，掌握脚本自动化测试和数据可视化技能，对未来进行复杂的系统性能分析将大有裨益。